



DOCDECK

The client document engine

A themed deck and product document for any project,
from chapters and live project data.

PREPARED FOR

Teams presenting software to their clients

EDITION

25 September 2026

Index

01	What docdeck is	3
	One set of chapters becomes a client deck and a product document, in the project's own look. Four files from one build The loop In numbers	
02	Setting up a project	5
	One command adds a docs folder; nothing is installed in the project. One command Nothing to install Commands	
03	Customising the look	6
	Map eight theme roles to the project's tokens and the whole document takes its colours, fonts and logo. Eight roles One config file Three levels of change	
04	Writing chapters	8
	Plain Markdown, one file per chapter, one slide per section. Blocks Deep in the book, brief in the deck	
05	Data and screenshots	10
	Tables are read from the project's code, and screens are captured from the running app. Providers Screenshots	
06	Keeping it current	11
	Chapters describe the product as it is today, and the check keeps history out. What the check catches Slides that fit Page numbers that are right	
07	The Claude Code skill	12
	Ask for the client deck in plain words; the skill writes the chapters and runs the engine. What the skill does Shaped to each project Installing	
08	What comes next	14
	The enhancements planned, in order of value.	

What docdeck is

One set of chapters becomes a client deck and a product document, in the project's own look.

Client meetings need two things: slides to talk over, and a document to leave behind. docdeck makes both from the same chapters, so they never disagree.

Four files from one build

16:9

deck.pdf

Slides to present or send.
One slide per section, text fitted to the slide.

Browser

deck.html

The same slides, presented with arrow keys and full screen.

A4

book.pdf

The full document: cover, index with page numbers, running footers, bookmarks.

Browser

book.html

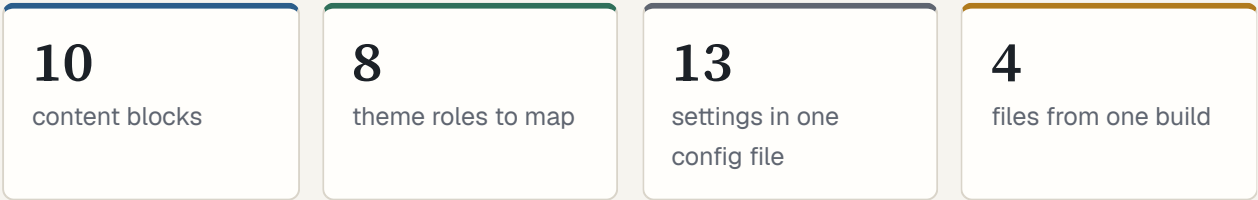
The same document as one page, index linked to each chapter.

The loop



When the product changes, you edit the chapter itself. There is no list of changes to maintain: each edition is the current truth, dated on the cover.

In numbers



Setting up a project

One command adds a docs folder; nothing is installed in the project.

One command

Run `docdeck init` in the project. It asks for the project’s name, its logo and the stylesheet that holds its colour tokens, and creates a small folder:

FILE	HOLDS
<code>docdeck.config.ts</code>	Name, logo, theme, chapters folder, screens
<code>providers.ts</code>	Data read from the project’s code
<code>GUIDE.md</code>	The project’s writing guide, followed by the skill
<code>chapters/NN-slug.md</code>	One chapter per file, in number order
<code>package.json</code>	Marks the folder as ES modules

Nothing to install

The engine lives in its own repository. The project’s config imports `docdeck` , and the engine answers that import itself, so the project’s dependencies stay untouched. A project wraps the command in a script, for example `pnpm docs:build` .

Commands

Command	Does
<code>docdeck init</code>	Creates the docs folder
<code>docdeck check</code>	Lints chapters, checks providers and screens, and fits every slide
<code>docdeck build</code>	Writes the four files; <code>--open</code> opens them, <code>--no-screens</code> reuses the last screenshots
<code>docdeck preview</code>	Serves the HTML and rebuilds on every save
<code>docdeck install-skill</code>	Installs the Claude Code skill, for the user or with <code>--project</code> for one project

Customising the look

Map eight theme roles to the project's tokens and the whole document takes its colours, fonts and logo.

Eight roles

Every colour in docdeck's templates is one of eight roles. A project maps its own design tokens onto them, and nothing else needs to change.

ROLE	USED FOR	NEUTRAL DEFAULT
paper	Page ground	#f7f7f5
ink	Text	#1a1d24
surface	Cards and tables	#ffffff
accent	Links, the current step, chapter rules	#2f4f7f
accent2	The other side of a comparison	#2e6b5e
attention	Decisions and warnings	#b0782b
rule	Hairlines	#d6d3cc
muted	Secondary text	#5d6270

One config file

SETTING	CHANGES
<code>root</code>	Where paths start
<code>project</code>	Name, title, tagline, audience, edition date
<code>brand</code>	Logo, logo on dark, footer mark
<code>theme</code>	Colour tokens and fonts
<code>chapters</code>	Chapters folder
<code>providers</code>	Data providers file
<code>templates</code>	Replacement templates
<code>screens</code>	App address, sign-in and screens to capture
<code>styles</code>	Extra stylesheets
<code>book</code>	Page size, margins, footer
<code>labels</code>	Words on the cover and index
<code>out</code>	Output folder
<code>outputs</code>	Deck, book, or both

Three levels of change

Settings

Colours, fonts, logo, page size, margins, footer and labels, all in the config.

Styles

An extra stylesheet for spacing, headings or table rules, using the theme roles.

Templates

Replace the cover, index or slide layout by copying the engine's templates into the project.

Writing chapters

Plain Markdown, one file per chapter, one slide per section.

A chapter is one Markdown file. Its number sets the order, its title and summary appear in the index, and each `##` section becomes a slide.

Blocks

Blocks drop real content into a chapter: tables and paths from the code, comparisons, screenshots, decisions.

BLOCK	SHOWS	USE FOR
<code>:::table</code>	Rows from a provider	Roles, categories, templates, statuses
<code>:::matrix</code>	A yes / no grid	Who can do what
<code>:::flow</code>	Numbered steps, with who acts	The path a record takes
<code>:::compare</code>	Two sides	One thing against another
<code>:::cards</code>	A grid of short items	Modules, account types, principles
<code>:::callout</code>	A highlighted note	Something to confirm, a limit
<code>:::screen</code>	A screenshot of the running app	Showing the real screen
<code>:::decisions</code>	Numbered questions for the client	What the client must decide
<code>:::columns</code>	Side-by-side prose	Two short explanations
<code>:::stats</code>	Three or four headline figures	Counts that make a point

Deep in the book, brief in the deck

The book

Everything, in order.

- Every section and every table
- Index with page numbers
- Running footer and bookmarks
- `:::book` detail included

The deck

The story, one idea a slide.

- One slide per section
- Text fitted to the slide
- `:::book` detail left out
- `:::deck` summaries added

Wrap detail in a `:::book` block and it stays out of the slides. Put a one-slide summary in `:::deck` and it stays out of the book.

Data and screenshots

Tables are read from the project's code, and screens are captured from the running app.

Providers

A providers file reads the project's own constants, seed data or API and hands tables, paths and cards to the chapters. When the product changes, the tables change with it at the next build.

A provider is data, or a function returning data. Chapters address it by path, for example `inward.path`. A missing provider fails the check, naming the chapter that asked for it.

Screenshots

The config gives the app's address, a sign-in, and the screens to capture. Each build signs in, opens every screen a chapter asks for, and places the capture in the slide and the page. When the app is not running, the last captures are reused.

Keeping it current

Chapters describe the product as it is today, and the check keeps history out.

What the check catches

CHECK FAILS ON	LEVEL
Looks like a commit hash; describe the change in the chapter instead	error
Mentions commits; clients read the current state, not history	error
Mentions a pull request	error
Mentions a changelog; update the chapter in place	error
Leftover TODO / FIXME / TBD	warning

Slides that fit

Every slide is laid out in the browser before the PDF is written. Text shrinks a little if it has to; a slide that still does not fit fails the check and names the section to split.

Page numbers that are right

The book prints each part on its own to count its pages, writes the index with the real page numbers, then prints the whole book once so the PDF carries bookmarks for every chapter and section.

The Claude Code skill

Ask for the client deck in plain words; the skill writes the chapters and runs the engine.

What the skill does

01

Read the config and GUIDE.md

Skill

02

Check the code for facts

Skill

03

Edit only the chapters asked for

Skill

04

Check, then build

Engine

05

Report outputs and contradictions

Skill

Shaped to each project

A project's `GUIDE.md` tells the skill who reads the document, which words to use, the chapter order and where the facts live in the code. For deeper changes, a project installs its own copy of the skill and edits its steps; that copy wins over the shared one.

Installing

- For every project: `docdeck install-skill` puts it in `~/.claude/skills/client-docs`.
- For one project: `docdeck install-skill --project` puts an editable copy in `.claude/skills/client-docs`, kept in the project's repository.
- After pulling a new engine, run the install again.

What comes next

The enhancements planned, in order of value.

NEXT	WHY
Publish to npm	Run it anywhere with npx
Build in CI	Every push rebuilds the PDFs
Speaker notes and presenter view	Present from the HTML deck
More languages and scripts	Marathi, Hindi and bilingual documents
Diagram and chart blocks	Flows and figures without screenshots
Annotated and mobile screens	Point at the part being described
Theme presets	A good look without design tokens
PowerPoint export	For clients who edit slides
Glossary and figure list	Long documents stay navigable
Edition summary	What changed since the last edition, without commit history